

Sample second midterm laboratory exam

INSTRUCTIONS

- Create a new workspace in your desktop
- **Import and change the name of the initial projects to surname_surname_name**

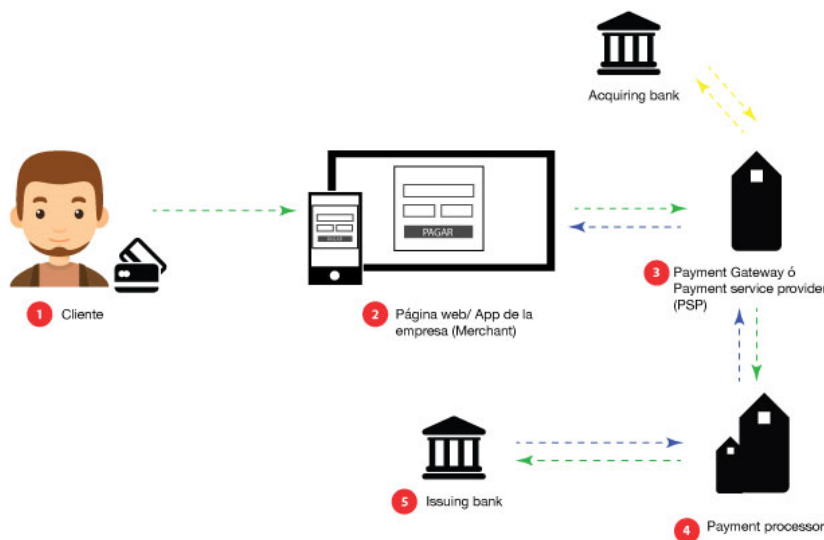
IT IS TOTALLY PROHIBITED

- USE CODE FROM OTHER PROJECTS (except mp.2021.util).
- Copy/send information through the network

Important remarks

1. You **MUST** draw a **UML class diagram FIRST**.
2. **Not only implementation**, but unit tests and exception handling, will be considered for grading.

1 Problem statement



A payment service provider (**PSP**) helps businesses to accept payments from customers. It connects businesses to financial entities (banks) to accept checks, credit/debit transactions, bank transfers,...

PAYPAY is a PSP and so it has an **application (payment processor)** to **validate money transactions made by customers to our clients (business)**. First, this application will **load information about transactions**. Then, app will **classify them into valid/invalid**. Finally, information about valid and invalid transactions is saved to different files.

Every **Transaction** is described in the application with the following information:

- **date of operation**. A string with the format yyyy/mm/dd. Example **2024/03/23**,
- **transaction mean identification**. The only accepted transaction means are credit cards and bank account transfers¹. Both are identified with a string,
- **amount** of the operation (a double) and
- **description** of the operation (a string).

A **CreditCardTransaction** also needs:

- **expiration date**. Credit card cannot be used after this date. A string with the format **yyyy/mm**, and
- **operation limit**. Maximum amount that can be charged to this credit card in a single transaction.

A **BankTransferTransaction** also stores the **account number** and the **type of client** that could be **Normal** or **Premium**.

2 Files format

2.1 Input file

Information about transactions comes in a plain text file with a name and extension **.pay**. This file contains one transaction per line, and it may contain lines with two different formats:

- Lines for credit card transactions:

```
cc;<op date>;<card number>;<exp date>;<operation limit>;<amount>;<description>
```

¹Transferencia bancaria



- Lines for bank account transfer transactions:

```
acc;<operation date>;<bank account number>;<client type>;<amount>;<description>
```

Where separator is semicolon (";"), client type is either **N** or **P** and other fields are as described above.

Sample file

```
cc;2021/05/03;5552468190471987;2021/12;500;400;Carrefour Travel Agency
cc;2021/05/05;6011898652586869;2021/12;1000;12;Valentin cafeteria
acc;2021/05/08;ES0310250124910021214445;N;150;Car repairing at Caris
acc;2021/05/08;ES0712452342121241551584;N;1000;Shopping at CFRD
```

2.2 Output files

Valid and invalid transactions will be written to different output files with the same name than source file.

- **File containing information about valid transactions** is a compressed file with extension **.valid.gz**.
- **File containing information about invalid transactions** is a plain text file with extension **.invalid.txt**.

3 Payment processing

The core of the application is the class **TransactionValidator** that will hold transactions **sorted by operation date and number** in ascending order (although transactions in the input file will be probably unsorted). It will examine each transaction and classify it as valid or invalid while recording every fault detected.

A transaction is deemed **invalid when it fails to meet any of the validation rules**. Each time the validator detects a fault, it will record a **failure code** together with the transaction. Please, mind that a transaction might **fail to meet more than one rule** and the application must record all faults.

3.1 Validation rules

Rules for bank account transfer transactions

- Account number must be a **valid IBAN**². Failure code **INVALID_IBAN**
- **Normal** clients can transfer 1000 € or less; there is no limit for **Premium** ones. Failure code **EXCEEDS_CLIENT_LIMIT**

Rules for credit card transactions

1. **Operation date** cannot be later than **expiration date**. Failure code: **EXPIRED_CARD**
2. **Amount** charged must be lower or equal **operation limit**. Failure code: **EXCEEDS_OPERATION_LIMIT**
3. Card number must be **Lhun**³ **valid**. Failure code: **INVALID_CARD**

Use class **IBAN** provided in package **uo.mp.transaction.model.util** to check if account number is valid IBAN

```
if ( IBAN.isValid( number ) ) ...
```

To check if credit card number is Lhun-valid use class **Lhun** provided in package **uo.mp.transaction.model.util**

```
if ( Lhun.isValid( number ) ) ...
```

To compare two dates just use **String.compareTo(...)** as usual. As dates are represented as **yyyy/mm/dd**, this method will work as we need and **2019/11/24 > 2019/11/12**

3.2 Saving valid and invalid transactions to files

Valid and invalid transactions will be saved in different output files, one transaction per line. Valid transactions will be saved **with the same format as the input file** while invalid transactions will **replace description by a concatenation of validation failure codes**. For example, let us consider the following line in the input file:

```
acc;2021/05/08;ES0;N;11250;Car repairing at Caris
```

if it was a valid transaction, the same line will be written to output file while, if it was invalid, the line would be

```
acc;2021/05/08;ES0;N;11250; EXCEEDS_OPERATION_LIMIT;INVALID_IBAN
```

² Internation Bank Account Number (IBAN)

³ https://en.wikipedia.org/wiki/Luhn_algorithm



3.3 Printing transactions to StandardOutput

Both, valid and invalid transactions, can also be displayed in standard output, in different lines, with the format:

- Lines for credit card transactions:

```
<op date>;cc;<card number>;<exp date>;<operation limit>;<amount>
```

- Lines for bank account transfer transactions:

```
<operation date>;acc;<bank account number>;<client type>;<amount>;<description>
```

Invalid transactions will also display the **number of faults detected**. For example, given this line in the input file

```
acc;2021/05/08;ES1234;N;11250;Car repairing at Caris
```

if the transaction was valid, the line to be displayed in the console would be

```
2021/05/08;ES1234;N;11250;acc;2 FAULTS
```

or, if it were a valid transaction

```
acc;2021/05/08;ES0;N;11250;acc
```

As Transactions are held sorted, they will also be written sorted to output files including standard output.

4 Error handling

Do not confuse the following errors with failing to meet transaction validation rules.

Do not consider any other errors except those described below. Assume that they will never happen and so they do not need to be controlled.

4.1 Argument validation

As usual, it is necessary to validate parameters in **every public method YOU implement**. It will be considered a programming error not meeting preconditions of the parameters and `IllegalArgumentException` is thrown, with an appropriate message, to indicate that a method has been passed an invalid argument.

- All arguments must be non-null, including lists.
- Strings must also be non-blank while numbers must be greater than zero.

4.2 Parsing errors

Only **invalid line format** is considered. An invalid line throws an `InvalidLineException` exception. These errors cause the line to be ignored, but continue parsing the argument, and log this exact message:

```
PARSING ERROR at line line-number: error-description
```

Lines are invalid when

- It is null or empty or blank; error-description will be NULL OR EMPTY LINE
- First field contains a string different from cc or acc; error-description will be INVALID KEYWORD
- Incorrect number of fields; error-description will be INCORRECT NUMBER OF FIELDS
- Operation limit and amount are not numbers; error-description will be INVALID NUMBER FORMAT
- Incorrect dates: not separated by /, not dd, mm and yyyy or mm and yyyy, any of dd, mm or yyyy are not numbers, dd is not in 1 to 31, mm is not in 1 to 12 and yyyy is before 2021. Error-description will be INVALID DATE FORMAT

4.3 User errors

They will **stop application** and **print**

```
ERROR Please mind this error: error-description
```

- Input file does not exist.** Error-description will be: input file cannot be found
- If, while adding transactions to `TransactionValidator`, it happens that **two transactions were made from the same transaction mean at the same date (same number and operation date)**, source file is considered corrupt. Error-description will be: input file is corrupt
 - Write a custom `TransactionException`, for application errors like those in section 4.3.



Write a custom `TransactionException`, for user errors.

4.4 Programming and system errors.

They make the program to stop. The following message must be printed in the standard output

```
PROGRAMMING ERROR Server stopped due to an internal error
```

and the exception stack trace will be recorded in the log.

5 Assessment

A working application DOES NOT mean an automatic pass. **You must have a minimum on:**

- Draw a UML class diagram for the application before start implementing. This first version might be improved as you implement the app. Submit only the final UML as an image in the final project. An initial project with some classes is provided. It gives some clues about the possible design. Please, be sure you understand it and check your UML against it.
- Exception handling
 - Throw and handle suitable exceptions to manage errors.
 - Print information in the standard output only in accepted classes.
- Input/Output StreamsInput/Output Streams
- Write Junit tests for the following methods
 - `TransactionValidator::validate()`
 - `Transaction::validate()` in all classes in the hierarchy
 - `TransactionLoader::Load()`. A file with incorrect lines is provided with the skeleton.

If you use your own collection classes you will be awarded with one extra point, added to the final mark of the exercise.